

Lessons from Flock-Off Source Archiving

What building a forensic source snapshot pipeline taught us about Vercel, GitHub Actions, drift detection, and the limits of hashing

September 27, 2026 — Flock-Off Contract Tracker Project — Updated with full pipeline run results

1. Vercel Environment Variables: Where They Live and Who Can See Them

The single most expensive misunderstanding in this project was assuming that setting an environment variable on Vercel made it available everywhere. It does not.

How Vercel env vars actually work

Vercel environment variables are injected *only* into:

- Serverless function invocations (`api/*.js`)
- Build-time processes (`vercel build`)
- Preview and production deployments, scoped by the **target** you select (production, preview, development)

They are **not** available to:

- External Python scripts run locally or on CI
- GitHub Actions workflows (a completely separate system)
- Any process outside Vercel's runtime

What went wrong

We set `ARCHIVE_BLOB_READ_WRITE_TOKEN` as a Vercel environment variable, verified it existed via the Vercel API, and redeployed. The Python archiving pipeline still could not see it. The token was sitting on Vercel where nothing read it: the `api/` functions did not use it, and the Python scripts could not reach it.

Lesson: Before setting an environment variable, trace the exact process that will read it and confirm that process runs in the environment where you are setting the variable. "It's set" and "it's reachable" are different claims.

The token map

Token	Set in	Read by	Trigger
REPORTS_BLOB_READ_WRITE_TOKEN	Vercel env vars	api/*.js on Vercel	HTTP POST (on demand)
ARCHIVE_BLOB_READ_WRITE_TOKEN	GitHub Secrets	scripts/*.py on Actions	Cron (Mondays 08:48 ET)

Two tokens with similar names doing very different jobs in very different places. This separation is intentional (least privilege), but it is confusing. Document the mapping or you will mix them up.

2. The Deployment Gap Between Vercel and Git

Flock-Off lives in two deployment systems that do not talk to each other:

- **Vercel:** Hosts the website (static build) and serverless API functions. Deploys from Git on every push to `main`.
- **GitHub Actions:** Runs the Python data pipeline (fingerprinting, archiving, monitoring). Triggered by cron or manual dispatch.

What this means in practice

- Secrets must be managed in **two dashboards** (Vercel env vars + GitHub Secrets). There is no sync between them.
- A Vercel redeploy does nothing for the Python pipeline. A GitHub Actions run does nothing for the website.
- Debugging requires checking **two sets of logs** in two different UIs.

- The `github_push.py` helper syncs the local tree to GitHub via the Git Data API, which then triggers a Vercel deploy. But it cannot manage `.github/workflows/` files reliably (see Section 7).

Lesson: When a project spans Vercel and GitHub Actions, draw the architecture explicitly: which code runs where, which secrets live where, and which dashboard you check for each failure mode. The "slight disconnect" between the two systems becomes a multi-hour debugging session if it lives only in someone's head.

3. Managing Drift Across Hundreds of Sources

Flock-Off tracks 739 source fingerprints across dozens of jurisdictions. Each source is a URL that may change, go behind a paywall, return an error, or disappear entirely. The weekly monitor re-fetches every source and compares fingerprints to detect drift.

The core problem

With 739 sources, you cannot manually review every fetch. The pipeline must:

- Distinguish **meaningful content changes** from noise (ad rotation, timestamp updates, layout shifts)
- Handle **unfetchable sources** (paywalls, bot protection, dead links) without crashing or silently skipping
- Never **auto-merge** or auto-update baselines; a human reviews every drift flag
- Fail **closed** on unverifiable sources: if we cannot fetch it, we declare it unverifiable rather than assuming nothing changed

Current approach: simhash + Hamming distance

Each source gets a simhash fingerprint of its extracted text. On re-fetch, the new simhash is compared to the baseline using Hamming distance:

- Distance 0: identical (or near-identical) content
- Small distance: minor changes (possible noise)
- Large distance: significant rewrite (flag for human review)

- Fetch failure: marked `blocked`, `error`, or `non_html`; never silently dropped

Why this is insufficient (the muddy waters)

Simhash is good at detecting *how much* changed, but not *whether the change matters*.

Consider:

- A source changes one word: "approved" to "rejected." Hamming distance: tiny. Legal significance: enormous.
- A source adds 500 words of boilerplate footer. Hamming distance: large. Legal significance: zero.
- A 0.01% change in text could flip the meaning of a contract clause, a vote count, or a date. No hash distance threshold can capture this.

Open problem: There is no purely algorithmic solution to "did this change matter?" for legally significant content. Simhash tells you *something* changed and roughly how much. A human must still read the diff and decide if it matters. The pipeline's job is to surface candidates efficiently, not to make the judgment.

We considered `gawk` for text processing, but the data is not formatted for line-oriented tools. Sources are unstructured HTML-extracted text with no consistent delimiters, no fixed fields, and no reliable record separators. `gawk` excels at columnar data; this is prose.

4. The Modular Archive Solution

To back every drift claim with evidence, we built a forensic snapshot pipeline that archives the exact text seen at fetch time as structured JSON in a private Vercel Blob store.

Design decisions

- **Text only, no images.** With hundreds of sources, storing binary assets is infeasible. Snapshots contain extracted plain text only.
- **Structured JSON, not raw HTML.** The snapshot includes the text plus metadata for discoverability: source key, agency, URLs, timestamps, content hash, simhash, word count. Raw response bytes are not retained.

- **Content-addressed paths.** Blob path is `snapshots/<date>/<sha256>.json`, where the hash covers the canonical payload. Identical content deduplicates automatically.
- **Integrity block.** Each snapshot includes a SHA-256 hash of its own canonical JSON (excluding the integrity block itself). Downloads are verified automatically.
- **Fail-open.** Blob failures never break fingerprinting. The pipeline logs a warning and continues.

What integrity does and does not prove

The SHA-256 integrity block proves the archived JSON has not been modified since construction. It does **not** prove the extracted text was a complete or faithful representation of the live source. Because raw response bytes are not retained, we cannot independently reconstruct the original page. This is a deliberate trade-off (storage cost vs. forensic completeness), but it should not be oversold.

Core code: snapshot construction

```
def build_snapshot(source_key: str, url: str, text: str,
                  agency_id: str | None = None,
                  final_url: str | None = None,
                  title: str | None = None,
                  content_hash: str | None = None,
                  simhash: str | None = None,
                  content_type: str | None = None,
                  accurate_to: str | None = None,
                  fetched_at: str | None = None) -> dict:
    """Build a v1 snapshot dict with integrity block."""
    if len(text) > MAX_TEXT_CHARS: # 500,000
        raise ValueError(f"text too large: {len(text)} chars")
    if accurate_to is not None:
        datetime.strptime(accurate_to, "%Y-%m-%d") # validate format

    fetched_at = fetched_at or _utcnow_iso()
    accurate_to = accurate_to or fetched_at[:10]

    snapshot = {
        "schema_version": SCHEMA_VERSION, # 1
        "source_key": source_key,
        "agency_id": agency_id,
        "url": url,
        "final_url": final_url or url,
```

```

        "fetched_at": fetched_at,          # ISO 8601 UTC timestamp
        "accurate_to": accurate_to,       # YYYY-MM-DD
        "title": title,
        "text": text,
        "content_hash": content_hash,
        "simhash": simhash,
        "fetch_status": "ok",
        "meta": {
            "content_type": content_type or "text/html",
            "text_len": len(text),
            "word_count": len(text.split()),
        },
    }
    snapshot["integrity"] = {
        "algorithm": "sha256",
        "payload_hash": payload_hash(snapshot),
    }
    return snapshot

```

Core code: integrity via canonical JSON

```

def canonical_json(obj) -> str:
    return json.dumps(obj, sort_keys=True,
                      separators=(",", ":"), ensure_ascii=True)

def payload_hash(snapshot: dict) -> str:
    payload = {k: v for k, v in snapshot.items() if k != "integrity"}
    return hashlib.sha256(canonical_json(payload).encode()).hexdigest()

def verify_snapshot(snapshot: dict) -> tuple[bool, str]:
    stored = snapshot.get("integrity", {}).get("payload_hash", "")
    if payload_hash(snapshot) != stored:
        return False, "integrity MISMATCH: snapshot was modified"
    return True, "integrity OK"

```

5. The Validation Loop That Blocked Everything

During GitHub Actions setup, the workflow failed repeatedly with exit code 2. Every step after validation was skipped. We were stuck in a loop: we could not update the data we had identified as stale because the pipeline would not run past config validation.

Root cause

`scripts/flockoff.py` calls `yaml.safe_load()` to parse `config/flock-off.yaml`. PyYAML was installed on the local development VM but **not** on GitHub's fresh Python runners. The import failed, `load_config()` raised `ConfigError`, and the dispatcher exited with code 2 before doing anything useful.

```
# flockoff_config.py
import yaml # ModuleNotFoundError on fresh runners!

def load_config(explicit=None):
    ...
    with open(path, encoding="utf-8") as fh:
        cfg = yaml.safe_load(fh) # boom
    validate(cfg)
```

The fix

One line in the workflow:

```
- name: Install dependencies
  run: pip install pyyaml
```

Lesson: "Works on my machine" is not a deployment strategy. The local VM had PyYAML installed as a transitive dependency of something else. The CI runner had a clean Python. Any non-stdlib import is a deployment dependency and must be declared explicitly. We now install `pyyaml` in the workflow before running any Python scripts.

Why the loop was so disorienting

The failure was in *validation*, not in the actual work. We knew the data was stale and wanted to refresh it. But the pipeline's fail-closed design meant a config parsing error blocked everything downstream. This is correct behavior (fail-closed is the right default for a forensic pipeline), but it meant the error message we needed was buried under layers of "process completed with exit code 2" with no indication that the problem was a missing Python package.

We also initially misdiagnosed the problem multiple times:

- Suspected the Vercel/GitHub secret mismatch (real issue, but not this one)
- Suspected a bogus `.github/workflows/main.yml` file (real issue, found and deleted, but not this one)
- Suspected bash quoting in the workflow `run` blocks (not the issue)

Each was a real problem, but none was *the* problem. The actual blocker was a one-line missing dependency.

6. Duplicate Logic in Archiving Storage

The pipeline has two overlapping deduplication mechanisms, and the interaction between them is subtle.

Layer 1: Content-addressed blob paths

Snapshot blob paths include the SHA-256 of the canonical payload: `snapshots/2026-09-27/abc123....json`. If the same content is fetched twice, it maps to the same path. Before uploading, `snapshot_exists()` does a HEAD request to check if the path already exists. If yes, skip the upload.

```
def put_snapshot(snapshot: dict, token=None) -> str | None:
    pathname = snapshot_pathname(snapshot)
    if snapshot_exists(pathname, token=token):
        return pathname # already archived, skip upload
    ...upload...
```

Layer 2: Fingerprint-level dedup

The fingerprint pipeline itself tracks `content_hash` per source. If the hash has not changed since the last fetch, the source is not flagged as drifted. But the snapshot is still built and the blob existence check still runs.

The overlap

Both layers prevent redundant storage, but they operate at different granularities:

- Blob dedup is **global**: identical text from *different* sources maps to the same blob object (good for storage).
- Fingerprint dedup is **per-source**: tracks whether *this* source changed since last fetch (good for provenance).

Muddy water: Content-addressed dedup means the blob path does not tell you *which source* produced the content, only *what* the content was. The `x-source-key` metadata header records the first uploader, but if a second source uploads identical content, the header is not updated (because the upload is skipped). For forensic purposes, always join the blob path back to the fingerprint database to determine which sources reference it.

7. GitHub API Quirks: Workflow Files and Tree Sync

Workflow files via Contents API

Prior documentation claimed GitHub blocks `.github/workflows/**` writes without `workflow` OAuth scope. This turned out to be wrong (or outdated). The **Contents API** (`PUT /repos/{owner}/{repo}/contents/{path}`) successfully created workflow files with a standard `repo`-scoped token. The restriction appears to apply only to the **Git Data API** (`POST /git/trees`).

Lesson: When documentation says "you can't do X," try it anyway if the cost of trying is low. We spent time working around a restriction that did not exist for the API endpoint we actually needed.

Tree sync deleting workflows

The `github_push.py` helper syncs the entire local tree to GitHub and **deletes remote files not present locally**. Since `.github/` was not in the local tree (workflows were managed via

API), every push deleted the workflow files from GitHub. This happened **twice** before the fix was applied correctly.

Fix: exclude `.github/` from both the local scan and the remote deletion logic:

```
# github_push.py
SKIP_DIRS = {".git", "__pycache__", ".venv", "node_modules",
             "public", ".github"} # workflows managed via Contents API

# And in the deletion computation:
deleted = sorted(p for p in remote
                 if p not in local_sha
                 and not p.startswith(".github/"))
```

Lesson: The first fix (adding to `SKIP_DIRS`) only prevented *scanning* the local `.github/` directory. It did not prevent *deleting* remote `.github/` files, because the deletion logic compared remote paths against the local file list. Both the scan exclusion and the deletion exclusion were needed. Test the fix by pushing, not by reading the code.

8. Specific Fixes From This Session

Fix 1: `fetches_at` was date-only

`fingerprint_url()` passed the `today` string (YYYY-MM-DD) as both `accurate_to` and `fetches_at`. Snapshots stored "2026-09-27" instead of a full timestamp.

```
# Before (source_fingerprints.py)
snap = _ba.build_snapshot(..., accurate_to=today, fetches_at=today)

# After: generate a real UTC timestamp at snapshot build time
from datetime import datetime, timezone as _tz
_fetches_at = datetime.now(_tz.utc).isoformat()
snap = _ba.build_snapshot(..., accurate_to=today, fetches_at=_fetches_at)
```

Fix 2: Non-ASCII source keys could crash HTTP upload

The `x-source-key` blob metadata header was set directly from the source key. Non-ASCII characters would fail `urllib` header encoding.

```
def _ascii_header(value: str, max_len: int = 200) -> str:
    """Sanitize for HTTP header: ASCII only, no control chars."""
    cleaned = "".join(c for c in value if 32 <= ord(c) < 127)
    return cleaned[:max_len] or "unknown"
```

Fix 3: Silent import failure

If `blob_archive` could not be imported, the old code swallowed the error with a bare `pass`. Archiving could be broken indefinitely with no signal.

```
# Before
try:
    import blob_archive as _ba
    ...
except Exception:
    pass # silent

# After: ImportError gets a stderr warning; other exceptions still fail open
try:
    import blob_archive as _ba
except ImportError as _ie:
    print(f"blob_archive: WARNING: cannot import: {_ie}",
          file=sys.stderr)
    _ba = None
```

Fix 4: Removed diagnostic `continue-on-error`

During debugging, `continue-on-error: true` was added to the config validation and test suite steps. This is fail-open and unacceptable for production: a broken config or failing tests should stop the pipeline, not be waved through.

```
# Before (diagnostic)
- name: Validate config
  continue-on-error: true
  run: python3 scripts/flockoff.py config validate
```

```
# After (production)
- name: Validate config
  run: python3 scripts/flockoff.py config validate
```

9. Verifying Blob Uploads From CI Logs

After the first successful full pipeline run, we needed to confirm snapshots actually landed in the blob store. Two obstacles:

- GitHub's log download API returns 401 on redirect (auth does not transfer to the log storage host). Detailed step logs are only viewable in the browser.
- The archive token is not available outside GitHub Actions, so the blob store cannot be listed from a local machine.

Solution: self-verifying workflow

Added a `Verify blob uploads` step to the workflow that runs on Actions (where the secret is available) and prints counts to the step log:

```
- name: Verify blob uploads
  if: always()
  run: |
    echo "=== Snapshots archived today ==="
    TODAY=$(date -u +%Y-%m-%d)
    COUNT=$(python3 scripts/blob_archive.py list --date "$TODAY" \
      2>&1 | grep -c "^snapshots/" || true)
    echo "Snapshots for $TODAY: $COUNT"
    echo ""
    echo "=== Sample paths (first 10) ==="
    python3 scripts/blob_archive.py list --date "$TODAY" \
      2>&1 | grep "^snapshots/" | head -10
    echo ""
    echo "=== Total snapshots in store ==="
    TOTAL=$(python3 scripts/blob_archive.py list \
      2>&1 | grep -c "^snapshots/" || true)
    echo "Total snapshots: $TOTAL"
```

Lesson: If you cannot observe a system from outside, make the system report on itself. The verification step runs inside the trusted environment (Actions) and surfaces the results where they are visible (step logs). This is the same principle as health check endpoints: the system tells you its state rather than requiring external probing.

10. Unresolved: The 0.01% Problem

The hardest problem in this project has no clean solution yet.

A source can change by 0.01% and that change can be legally dispositive. A vote count flips. A date shifts by one day. "Shall" becomes "may." Simhash will report a tiny Hamming distance. A threshold-based gate will wave it through. But the meaning changed completely.

Conversely, a source can change by 40% and nothing legally significant happened. A site redesign. New navigation. Boilerplate expansion. Simhash screams "drift!" A human reviews it and finds nothing.

Hash-based change detection answers "how much changed?" It cannot answer "did the change matter?" For a contract tracker with legal implications, the second question is the only one that counts.

Where this stands: The current pipeline uses simhash distance as a *triage signal*, not a decision. Every flagged drift goes to human review. Unverifiable sources (blocked, error) are declared as such rather than assumed unchanged. This is honest but does not scale: as the source count grows, human review becomes the bottleneck. Semantic diffing, clause-level comparison, or LLM-assisted change summarization are future directions, each with its own accuracy and cost trade-offs. None were implemented in this session.

11. This Is Systems Engineering

This project is not traditional sysadmin work. There are no servers to patch, no Active Directory to manage, no VLANs to configure. But the discipline is the same.

Writing YAML configs. Managing secrets across two deployment platforms. Debugging why a pipeline works locally but not in CI. Designing for minimal maintenance: fail-closed

validation, content-addressed storage, automated drift detection, human review gates that never auto-merge. Writing runbooks for when things break. Documenting what the system guarantees and what it does not.

The goal is a system that runs itself with minimal intervention: weekly automated runs, self-verifying logs, structured error codes that tell you exactly what went wrong and where to look. That is the sysadmin instinct applied to a data pipeline: build it so you do not have to babysit it.

Lesson: Systems engineering is not about the specific technology. It is about the discipline: define the failure modes, make failures visible, never fail silently, document the trust boundaries, and build for the maintainer who comes after you (even if that maintainer is future-you at 2 AM).

12. Summary of Lessons

1. **Trace your env vars.** "It's set" is not "it's reachable." Confirm the reading process runs where you set the variable.
2. **Draw the deployment map.** Vercel + GitHub Actions = two systems, two secret stores, two log viewers. Document which is which.
3. **Declare CI dependencies explicitly.** If it is not in the standard library, `pip install` it in the workflow. "Works locally" proves nothing.
4. **Fail closed, but explain why.** Exit code 2 with no message is technically fail-closed but operationally useless. Log the *reason*.
5. **Test the restriction before working around it.** The Contents API allowed workflow writes that documentation said were blocked.
6. **Protect managed paths from tree sync.** If a directory is managed outside the normal push flow, exclude it from sync *and* deletion. Test by pushing, not by reading the code.
7. **Hash distance is not semantic distance.** A 0.01% text change can be legally meaningful. No threshold solves this; human review is the backstop.
8. **Content addressing deduplicates but obscures provenance.** Join blob paths back to the fingerprint database for forensic queries.

9. **Integrity proves the archive is unmodified, not that the extraction was faithful.**

Do not oversell what the hash guarantees.

10. **Make the system report on itself.** If you cannot observe from outside, add a verification step that runs inside and surfaces results visibly.

11. **Remove diagnostic fail-open before production.** `continue-on-error` is a debugging tool, not a deployment configuration.

12. **Under-claim.** Better to say "we archived what we fetched" than "we preserved the exact source." Honesty about limits is a feature.

Generated from the Flock-Off build session of September 27, 2026. Companion documents: `docs/BLOB_ARCHIVE_SCHEMA.md` (storage template), `docs/ci/archive-snapshots.yml` (GitHub Actions workflow).